



# IMP – Zabezpečovací systém s detekcí pohybu

Matej Tomko  
xtomko06

13. decembra 2023

Odkaz na video s demonštráciou projektu

## Obsah

|          |                                     |          |
|----------|-------------------------------------|----------|
| <b>1</b> | <b>Úvod do problému</b>             | <b>2</b> |
| 1.1      | Hardvér . . . . .                   | 2        |
| 1.2      | Softvér . . . . .                   | 2        |
| <b>2</b> | <b>Popis riešenia</b>               | <b>2</b> |
| 2.1      | Využité prostriedky . . . . .       | 3        |
| 2.2      | Implementácia . . . . .             | 4        |
| 2.2.1    | Senzor pohybu . . . . .             | 4        |
| 2.2.2    | Fotobunka s HTTP serverom . . . . . | 4        |
| <b>3</b> | <b>Práca s webovou stránkou</b>     | <b>5</b> |
| <b>4</b> | <b>Nedostatky Implementácie</b>     | <b>6</b> |

## Úvod do problému

Úlohou projektu bolo zostrojiť koncept zabezpečovacieho systému s detekciou pohybu pre stráženie prístupu do stanovených priestorov v interiéri s využitím vývojového kitu na báze SoC(system-on-chip) ESP32 od firmy Espressif a potrebných externých senzorov

## Hardvér

Na riešenie som mal možnosť použiť vývojové kity Espressif ESP32 ESP-EYE a ESP32 WEMOS D1 R32. Pohybový senzor je PIR Sensor(passive infrared sensor).

## Softvér

Pri výbere softvéru na programovanie hardvéru som nemal žiadne zábrany a rozhodoval som sa medzi jazykom Nim s knižnicou Nesper, Arduino a ESP-IDF.

- Knižnica Nesper je wrapper pre ESP-IDF a je napísaná v jazyku Nim, čo je staticky typovaný prekladaný programovací jazyk so syntaxom podobným Pythonu a rýchlosťou porovnateľnou s jazykom C. Výsledok jeho prekladu je v našom prípade zdrojový kód v jazyku C alebo v jazyku C++, ktorý je následne preložený zvoleným prekladačom. Toto ho robí vhodný pre programovanie aj na embedded platformách.
- Knižnice Arduino sú priechinky, ktoré združujú viac súborov so zdrojovými kódmi. Vďaka týmto zdrojovým kódom dokážeme potom jednoduchšie a prehľadnejšie vytvoriť vlastný program, keď chceme pripojiť nejaký zložitejší modul, senzor, displej a podobne.
- ESP-IDF je oficiálny framework od firmy Espressif písaný v jazyku C.

Na riešenie som si vybral Arduino[1], pretože má dostatočne veľkú podporu a rovnakú funkcionality ako ESP-IDF. Riešenie projektu pomocou Arduino je podobne ako ESP-IDF riešené štýlom jazyka C. Po preskúmaní knižnice Nesper pre jazyk Nim som zistil, že je pomerne stabilný, no nemá dostatočnú dokumentáciu a na prácu je potrebné dostatočne poznať ESP-IDF.

## Popis riešenia

Zo začiatku som plánoval uľahčiť si prácu tým, že by som vytvoril svoje riešenie pomocou jedného modulu ESP-EYE, ktorý by slúžil zároveň ako senzor, fotobunka a HTTP server, pomocou ktorého sa budú dáť snímky zobrazíť vo webovom prehliadači. Problém nastal po zistení, že toto riešenie nie je možné, pretože daný modul nemá rozhranie GPIO(general-purpose input/output), na ktoré by som mohol PIR senzor pripojiť.

Preto som sa rozhodol na riešenie použiť 2 moduly ESP32 ako už zo zadania vyplýva. ESP-EYE modul v roli kamery a HTTP serveru a ESP32 WEMOS D1 R32, ktorý má rozhranie GPIO a teda je ku nemu možné pripojiť PIR senzor na detekciu pohybu v roli klienta, ktorý sa na daný vytvorený server pripája a posiela dáta o pohybe pred senzorom.

Na týchto zariadeniach bežia 2 rôzne programy a komunikujú pomocou knižnice *WiFi.h*, na serveri *ESPAsyncWebServer.h* na klientovi *HTTPClient.h*.

## Využitie prostriedky

Na riešenie projektu bol využitý nasledovný hardvér:

- modul ESP-EYE ESP32-CAM[2][3]
- ESP32 WEMOS D1 R32[4]
- PIR senzor[5]

Softvér je písaný v jazyku *C* s využitím nasledujúcich knižníc:

- WiFi.h:
  - Účel: Umožňuje ESP32 pripojiť sa k bezdrôtovým sieťam.
  - Použitie: Používa sa na konfiguráciu a správu Wi-Fi pripojení na ESP32.
- esp\_camera.h:
  - Účel: Poskytuje funkcie a nástroje na komunikáciu s kamerami pomocou ESP32.
  - Použitie: Umožňuje zachytávať obrázky alebo videá zo kamier pripojených k ESP32.
- soc/soc.h a soc/rtc\_cntl\_reg.h:
  - Účel: Súčasť knižnice pre System-on-Chip (SoC) ESP32.
  - Použitie: Používa sa na konfiguráciu rôznych nastavení súvisiacich s čipom ESP32, vrátane riadenia hodín v reálnom čase (RTC).
- driver/rtc\_io.h:
  - Účel: Súčasť knižnice pre RTC (hodiny v reálnom čase) na ESP32.
  - Použitie: Umožňuje riadiť a konfigurovať vstupné a výstupné piny RTC.
- esp\_timer.h:
  - Účel: Poskytuje funkcionality pre časovače na ESP32.
  - Použitie: Používa sa na spracovanie a riadenie časovačov v aplikáciách ESP32.
- img\_converters.h:
  - Účel: Funkcie pre konverziu obrázkov na ESP32.
  - Použitie: Poskytuje nástroje na konverziu medzi rôznymi formátmi obrázkov.
- ESPAsyncWebServer.h[6][7]:
  - Účel: Asynchrónna HTTP a WebSocket server knižnica pre ESP32.
  - Použitie: Umožňuje vytvárať webové servery na ESP32, ktoré môžu spracovávať viacero požiadaviek súčasne.
- Arduino.h:
  - Účel: Súčasť Arduino jadra pre ESP32.
  - Použitie: Poskytuje základné funkcie a definície Arduino pre vývoj na ESP32.

- SPIFFS.h a FS.h:
  - Účel: Knižnice súborového systému pre ESP32.
  - Použitie: Umožňujú čítať a zapisovať súbory do Flash súborového systému ESP32 (SPIFFS - SPI Flash File System).
- HTTPClient.h:
  - Účel: HTTP klient knižnica pre ESP32.
  - Použitie: Umožňuje ESP32 vykonávať HTTP požiadavky, ako napríklad získavanie údajov zo servera pomocou HTTP.

## Implementácia

### Senzor pohybu

Kód pre ESP32 WEMOS D1 R32 slúži na detekciu pohybu pomocou PIR senzora a následne odoslanie HTTP požiadavky na zadanú URL adresu. Keďže ide o časť projektu týkajúceho sa IoT (Internet of Things), kód kombinuje prácu s bezdrôtovým pripojením cez Wi-Fi a základné riadenie digitálnych vstupov/výstupov.

- Príprava pripojenia k Wi-Fi:
  - V úvode kódu sú definované údaje pre pripojenie k bezdrôtovej sieti Wi-Fi, vrátane názvu siete (SSID) a hesla.
- Inicializácia a pripojenie k Wi-Fi:
  - V metóde setup() sa inicializuje sériová komunikácia a konfiguruje pin PIR senzora ako digitálny vstup. Následne sa pripája k Wi-Fi sieti.
- Detekcia pohybu:
  - V nekonečnom cykle metódy loop() sa číta hodnota PIR senzora. Ak senzor detekuje pohyb (hodnota log. 1), je vypísaná správa o detekcii a vytvárajú sa inštancie tried WiFiClient a HTTPClient.
- Odoslanie HTTP požiadavky:
  - V prípade detekcie pohybu sa vytvára HTTP požiadavka typu GET na zadanú URL adresu. Táto adresa odkazuje na server alebo službu, ktorá sa stará o spracovanie a ukladanie obrázkov alebo dát spojených s detekciou pohybu.

Používateľ môže prispôbiť kód podľa svojich potrieb, najmä pokiaľ ide o adresu Wi-Fi siete, heslo a cieľovú URL adresu pre odosielanie HTTP požiadavky.

### Fotobunka s HTTP serverom

Kód pre ESP32-CAM implementuje možnosť snímania fotografií. Využíva bezdrôtové pripojenie cez Wi-Fi a obsahuje webový server, ktorý umožňuje ovládať systém a zobrazovať zachytené snímky cez webový prehliadač.

Kľúčové časti kódu:

- Pripojenie k WiFi:
  - Nastavuje sa pripojenie k bezdrôtovej sieti so zadaným SSID a heslom.
- Inicializácia kamery:
  - Nastavuje sa konfigurácia kamery vrátane pinov a parametrov obrazu.
- Web server:
  - Vytvára sa inštancia AsyncWebServer s pripojením na port 80.
  - Definujú sa rôzne cesty pre obsluhu HTTP požiadaviek, vrátane základnej stránky "/", požiadavky na snímame "/capture" a požiadavky na zobrazenie zachytených fotografií "/saved-photo".
- Snímanie a ukladanie fotografií:
  - Funkcia capturePhoto sníma fotografiu pomocou kamery, rozsvieti zabudovanú LED diódu, a uloží zachytený snímok do pamäte SPIFFS s názvom "/photo.jpg".
- Kontrola existencie fotografií:
  - Funkcia checkPhoto kontroluje, či existuje uložená fotografia vo SPIFFS.
- Nekonečný cyklus loop:
  - Kontroluje sa príznak takeNewPhoto, a ak je nastavený, vykoná sa snímame fotografií.
  - Kód je navrhnutý tak, aby stále snímá nové fotografií, pokiaľ nebude fotografia úspešne uložená.

## Práca s webovou stránkou

Kód HTML predstavuje webovú stránku pre ovládanie zabezpečovacieho systému s detekciou pohybu. Používatelia môžu na tejto stránke vykonávať nasledujúce činnosti:

- Zachytiť Fotografiu:
  - Po stlačení tlačidla "SNÍMKA FOTOGRAFIE" sa vyšle požiadavka na server, ktorá spustí proces zachytenia fotky pomocou kamery na ESP32-CAM.
  - V takomto prípade nemusí pohybový senzor zachytiť pohyb.
- Otočiť Fotografiu:
  - Stlačením tlačidla "OTOČIŤ" používatelia môžu otáčať zobrazenú fotografiu o 90 stupňov v smere hodinových ručičiek. Táto funkcionality môže byť užitočná, ak je fotografia nesprávne orientovaná.
- Obnoviť Stránku:
  - Tlačidlom "OBNOVIŤ STRÁNKU" môžu používatelia obnoviť webovú stránku, čo zabezpečí aktualizáciu a zobrazenie najnovšej fotografie.

- Automatika Snímania Fotografie:
  - Systém automaticky zachytí fotografiu, keď detekuje pohyb pomocou PIR senzora. Používatelia môžu pozorovať zachytené snímky na webovej stránke.

Stránka je prezentovaná jednoduchým a prehľadným spôsobom, využíva štýly pre príjemné prostredie a umožňuje používateľom interagovať s ESP32-CAM pre snímanie a manipuláciu s fotografiou.

## Nedostatky Implementácie

Hlavným nedostatkom je fakt, že od zachytenia pohybu pred pohybovým senzorom až po nahranie obrázku na webovú stránku môžu v extrémnych prípadoch prejsť až 4 sekundy.

Ďalší nedostatok, ktorého som si po implementácii projektu všimol je, že stránku, ktorá zobrazuje zachytené obrázky, musí užívateľ ručne znova načítať.

## Literatúra

- [1] Arduino, “Arduino software,” <https://www.arduino.cc/en/software>, 2023.
- [2] H. spol. s.r.o., “ESP32-CAM, 2,4GHz WiFi+Bluetooth modul+kamera OV2640,” <https://www.hadex.cz/m432c-esp32-cam-24ghz-wifiblueetooth-modulkamera-ov2640/>, 2023.
- [3] —, “Vývojová deska pro modul ESP32-CAM,” <https://www.hadex.cz/m432m-vyvojova-deska-pro-modul-esp32-cam/>, 2023.
- [4] I. V. Šimek, “Schéma zapojení desky Wemos D1 R32,” [https://www.fit.vutbr.cz/~simekv/IMP\\_projekt\\_board\\_ESP32\\_Wemos\\_D1\\_R32.pdf](https://www.fit.vutbr.cz/~simekv/IMP_projekt_board_ESP32_Wemos_D1_R32.pdf), 2023.
- [5] H. spol. s.r.o., “Detektor pohybu, modul PIR SB00312A-1,” <https://www.hadex.cz/m356-detektor-pohybu-modul-pir-sb00312a-1/>, 2023.
- [6] M. N. Dev, “ESPAsyncWebServer,” <https://github.com/me-no-dev/ESPAsyncWebServer>, 2022.
- [7] —, “AsyncTCP,” <https://github.com/me-no-dev/AsyncTCP>, 2019.